SANPO USB 转 RS485

English

SANPO USB 转 RS485 协议用于在 Windows 和 Linux 上控制 RS485 设备，RS485 数据转发给设备，无需驱动。发送报文在 原始RS485 数据帧 外附加 SANPO 自定义帧头/尾，附加帧只作为开发板识别使用，下发给 RS485 设备的只有原始 RS485 数据，不会加入 SANPO 帧头/尾。

连接和通信参数

接入开发板USB后会出现 6 个 USB 串口，其中 2 个是管理口，另外 4 个分别对应四路 RS485 透传串口。请注意本协议模式下发送RS485数据，使用两个管理口，不是4个RS485透传串口(4个透传串口用于 RS485原始透传模式)。2个管理口一个访问 RS485-1/2，另一个访问 RS485-3/4。首次使用时建议只连接一个设备，确认 USB 串口和物理接口的对应关系。Linux 命令模式下，也可以按照串口识别工具使用说明运行 Python 程序，识别哪 4 个串口是 RS485 串口。

出厂默认 RS485 参数为 4 Mbps、1 个停止位、无校验、8 个数据位。但是某些 Linux 串口枚举会改变当前运行中的参数，因此，协议模式下建议每次发送业务数据前，显式发送 `AT+SETRS485` 重设 RS485 参数。详情参考 通信参数配置。

发送 `AT+` 命令时使用文本模式，并把发送结束符设置为 `CRLF`。不要把字符 `\r\n` 当作普通文本发送。

```
AT+SETRS485=<波特率>,<停止位>,<校验>,<数据位>
```

例如，设置为宇树电机常用的 4 Mbps、8-N-1：

```
AT+SETRS485=4000000,1,0,8
```

示例程序：发送数据前设置 RS485 波特率

安装依赖：

```
python -m pip install pyserial
```

下面的示例代码会自动找到并通过2个管理口设置 RS485-1～RS485-4 的波特率。示例设置为宇树电机常用的 4 Mbps、8-N-1。

```
import re
import time
import serial
from serial.tools import list_ports

COMMANDS = ["AT+SETRS485=4000000,1,0,8"]

def is_management_port(port):
    if (port.vid, port.pid) != (0x1209, 0x2323):
        return False
    info = " ".join(str(getattr(port, name, "") or "")
                    for name in ("hwid", "interface", "location")).upper()
    return ("MI_01" in info or "MANAGEMENT PORT" in info or
            re.search(r":(?:X|\d+)\.1(?:\D|$)", info) is not None)

def send_at(stream, command):
    stream.reset_input_buffer()
    stream.write(command.encode("ascii") + b"\r\n")
    stream.flush()
    deadline, reply = time.monotonic() + 1.0, bytearray()
    while time.monotonic() < deadline:
        reply.extend(stream.read(stream.in_waiting or 1))
        if b"OK\r\n" in reply or b"ERR," in reply:
            break
    text = reply.decode("ascii", errors="replace").strip()
    if "OK" not in text:
        raise RuntimeError(f"{stream.port}: {command}: {text or 'no reply'}")
    print(f"{stream.port}: {command} -> {text}")

ports = sorted(p.device for p in list_ports.comports()
               if is_management_port(p))
if len(ports) != 2:
    raise SystemExit(f"Expected 2 SANPO management ports, found {len(ports)}:
{ports}")

for device in ports:
    with serial.Serial(device, 1_000_000, timeout=0.05, write_timeout=1) as stream:
        time.sleep(0.05)
        for command in COMMANDS:
            send_at(stream, command)

print("RS485-1..RS485-4 configured successfully.")
```

需要其他串口参数时，只修改 `COMMANDS` 中的四个数字。代码中打开管理口使用的 `1_000_000` 只是 CDC API 参数，不会设置 CAN 或 RS485 的物理波特率；物理 RS485 参数只由明确 的 `AT+SETRS485` 命令修改。

参数	支持值
波特率	`2400` ～ `4000000`，`5000000`，`6000000`，`7500000`
停止位	`1` 或 `2`
校验	`0` 无校验；`1` 奇校验；`2` 偶校验
数据位	`8` 或 `9`

封装报文格式

发送和接收使用相同的外层格式：

字段	长度	说明
帧头	2 字节	固定 `52 54`，即 `RT`
Channel	1 字节	RS485 接口编号
数据长度	1 字节	原始 RS485 数据长度，最大 64 字节
数据	0～64 字节	设备厂家定义的原始指令或反馈
帧尾	2 字节	固定 `0D 0A`

开发板向 RS485 总线发送时会去掉 `RT`、Channel、长度和帧尾，只发送中间的原始数据。设备反馈时，开发板重新添加 `RT` 外层，并在 Channel 中标出数据来自本地第一路还是第二路。

宇树 GO-M8010 示例

宇树原始控制帧：

```
FE EE 11 00 00 00 00 00 00 00 00 00 00 00 18 7C
```

将这 17 字节数据发送到 Channel 2：

```
52 54 02 11 FE EE 11 00 00 00 00 00 00 00 00 00 00 00 18 7C 0D 0A
```

其中 `02` 是目标接口，`11` 是十六进制的 17。RS485 总线上实际发送的仍然只是以 `FE EE` 开头的 17 字节宇树指令。

电机反馈会类似下面的形式返回：

```
52 54 02 10 FD EE ...共16字节电机反馈... 0D 0A
```

完整示例：USB 转 RS485 宇树 GM8010 使用说明

该示例每次打开管理口后都会先强制发送 `AT+SETRS485=4000000,1,0,8`，再发送电机报文。这样即使独立 RS485 透传串口此前用其他波特率打开并改变了当前运行中的 UART 参数，协议模式也会 恢复为宇树 GM8010 所需的 4 Mbps、8-N-1。相同配置会重新应用到当前 MCU 的两路 RS485，但不会重复写入 Flash。

Windows 示例：

```
python usb2rs485_unitree_demo_v8.py --port COM9 --motors 1 --channel 2
```

Linux 示例：

```
sudo python3 usb2rs485_unitree_demo_v8.py --port /dev/ttyACM0 --motors 1 --channel 2
```

回包处理

一次 RS485 接收事件对应“一段连续收到的数据”，不保证正好是一条完整的电机协议帧。Channel 可以确认数据来自哪一路接口，但上位机仍应按照电机协议的帧头、长度和校验重组完整反馈。

USB 同样是字节流，一次读取可能得到半帧或多帧。解析时应先找到 `52 54`，读取长度字段，再等待完整的 `6 + 数据长度` 字节。

常见问题

现象	处理方法
指令被发到两路 RS485	Channel 为 `0x00`；改为具体通道
发送后没有反馈	检查 USB 串口、Channel、RS485 参数、设备 ID、指令校验、A/B 接线和供电
`AT+SETRS485?` 显示 4 Mbps，但协议模式仍无反馈	保存值可能正确，但独立透传口曾改变当前 UART 参数；重新发送 `AT+SETRS485=4000000,1,0,8`。本页 GM8010 示例会自动执行
回包 Channel 显示 1/2 而不是 3/4	回包使用当前 MCU 的本地编号，属于正常行为

现象	处理方法
回包被拆成多段	按 `RT` 长度字段重组外层帧，再按电机协议解析 Data